University of Salford, MSc Data Science

Module: Machine Learning & Data Mining

Session: Workshop Week 7

Topic: Feature Selection & Class Imbalance

Tools: Jupyter Notebook

Objectives:

After completing this workshop, you will be able to:

- Use filter methods for feature selection (using the VarianceThreshold and SelectKBest selectors in Scikit Learn)
- Use wrapper methods for feature selection (such as Recursive Feature Elimination)
- Understand the problem of class imbalance and use the imblearn package for resampling, including oversampling, undersampling and SMOTE.
- Use the imshow function from matplotlib to visualise images stored as a Numpy array

MLDM Week 7 Workshop: Feature Selection & Class Imbalance

University of Salford MANCHESTER

Introduction:

Feature selection is the process of selecting the features in the data which are most useful to the problem you are working on. For example, when training a classification or regression model, not all the features within the data may be useful or relevant to making a prediction. This can negatively impact the model in several ways:

- Including features which aren't relevant introduces more 'noise' into the data which can ultimately make it *less* accurate than a model trained on fewer features.
- Having a high number of features (particularly if the number of instances in your training data is low) increases the risk of overfitting – this is known as the 'curse of dimensionality'.
- Also in a real-world setting, collecting the data itself may take time and effort. Identifying which features are relevant and only including those in the model could therefore make it easier and cheaper to collect the data needed to use the model.
- Including more features increases the training time.

Some models are more sensitive than others. For example, K-Nearest Neighbours and Neural Networks can be very sensitive to irrelevant features, while logistic regression is sensitive to features which are highly correlated.

There are three main approaches to feature selection:

- **Filter methods** use a statistical measure to rank the features contained within the data. Features not meeting a pre-set threshold can then be eliminated. Alternatively, the best *K* features can be selected.

- **Wrapper methods** treat feature selection as a search problem and use a model to evaluate the highest performing combination of features.
- **Embedded methods** learn which features contribute most to the model accuracy as part of the model training process itself. In other words, embedded methods don't require a separate feature selection step because this is performed as an integral part of the model training.

The aim of the workshop is to explore various approaches to feature selection and provide an intuitive understanding of how and why they work.

The Dataset:

The dataset we are using for this workshop is the MNIST handwritten digits database. This is a widely used image dataset, which includes 70,000 low resolution images of handwritten digits 0-9, with the class label specifying which digit it is. This can therefore be used for multiclass classification problems. Most typically, this dataset can be used to train deep learning models, however, we will be using it to demonstrate feature selection. Training a model which can recognise handwritten text and digits has broad applications – from digitisation of historical archives through to automating repetitive data entry tasks.

In the MNIST dataset, each image is a greyscale image of 28x28 pixels. Each pixel can take a value between 0 and 255, where 0 is a white pixel and 255 is a black pixel. Each image can therefore be represented as a total of 784 (28x28) values, each one representing one of the pixels in the image. Each of these 784-pixel values can be thought of as a feature which we can use to train a classification algorithm. This dataset therefore has high dimensionality, as we have 784 features in total!

For the purposes of this workshop, we have provided a smaller sample from the full database saved on Blackboard as **MNIST_Shortened.csv**. The data is provided in a tabular format with one row per image. There are then 785 columns, the first 784 holding the pixel values and the final column holding the class label. In total, there are 6,000 images in our extract from the MNIST database. There are 10 classes (one each for the digits 0 to 9).

Part One: Exploring the Data

1) Start Jupyter Notebook, and in the browser navigate to the directory you want to save this week's workshop notebook in. The select New from the right-hand side and select Notebook 3 to open a new notebook.

Download the **MNIST_Shortened.csv** file and save it in the same directory.

2) In your notebook, first run the imports that are needed:

```
# Run the imports we need
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
```

3) Now we can use the genfromtxt function to create a Numpy array from the csv file you have just downloaded from Blackboard. We can then define X and y from this Numpy array (remember, the first 784 columns are the features, and the last column is the class label)

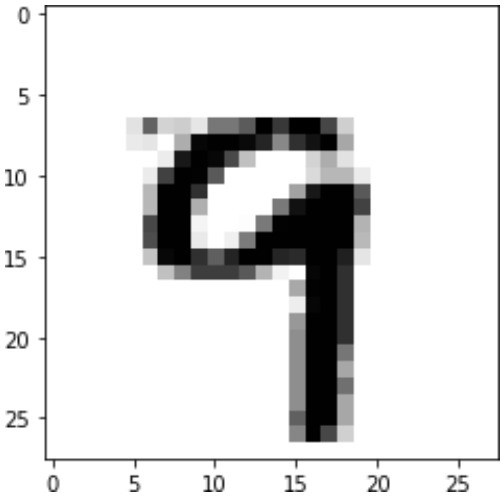
```
mnist = np.genfromtxt('MNIST_Shortened.csv', delimiter=',',
skip_header=1)
# Define X and y

X = mnist[:,0:784]
y = mnist[:,-1]

# Check dimensions of X

X.shape
```

You should see that X has dimensions (6000, 784).

4) Because this data is image data, to explore it we are going to visualise some of the handwritten digits. To do this, we can select one row of the array, and then re-shape it so that it is a 2-dimensional array of shape 28x28, recreating the original image. We can then use the imshow function within Matplotlib to show this as an image.

```
# we can use the Numpy reshape function along with the matplotlib
imshow function to visualise

plt.imshow(X[0].reshape(28,28),cmap='gray_r')
plt.show()
```

The cell output should show this image:



5) We can view more images at a time if we loop through a range. Here we use the Numpy randint function to select 100 images at random from the dataset and display these using imshow.

```
plt.figure(figsize=(8, 12))

for i in range(100):

    plt.subplot(10,10,i+1)
    plt.imshow(X[np.random.randint(0,6000)].reshape(28,28),cmap='gray_r')
    plt.axis('off')
plt.show()
```

In the cell output, you should see something similar to the below (The digits shown here won't be the same as we have selected these randomly)

6) Use the train_test_split function to split the data into a training dataset and a test dataset.

```
# Split data into train and test sets

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=0, stratify=y)
```

Part Two: Using VarianceThreshold to remove low or no variance features

7) We are first going to use the VarianceThreshold selector from Scikit Learn to remove any features which have no variance. This is a filter method which simply filters out those features which have low variance, below a threshold you can set. Unlike most filter methods, this looks only at the values of the features, rather than computing test statistics using class labels. We can use this as a simple baseline approach to feature selection, because features with very little or no variance do not contribute much useful information to the model.

We first instantiate the selector and set the variance threshold we want to use. We will set this to 0, which means we are just removing the features which have no variance at all. These features are entirely redundant because they take the same value for every single observation in the training set.

We then use the fit_transform function on the training data, which identifies the features below the variance threshold and transforms the array to remove them. Finally, we use the transform function on the fitted selector to carry out the same transformation to the test dataset.

```
from sklearn.feature_selection import VarianceThreshold

variance_selector = VarianceThreshold(threshold=0)

X_train_fs = variance_selector.fit_transform(X_train)
X_test_fs = variance_selector.transform(X_test)

print(f"{X_train.shape[1]-X_train_fs.shape[1]} features have been
removed, {X_train_fs.shape[1]} features remain")
```

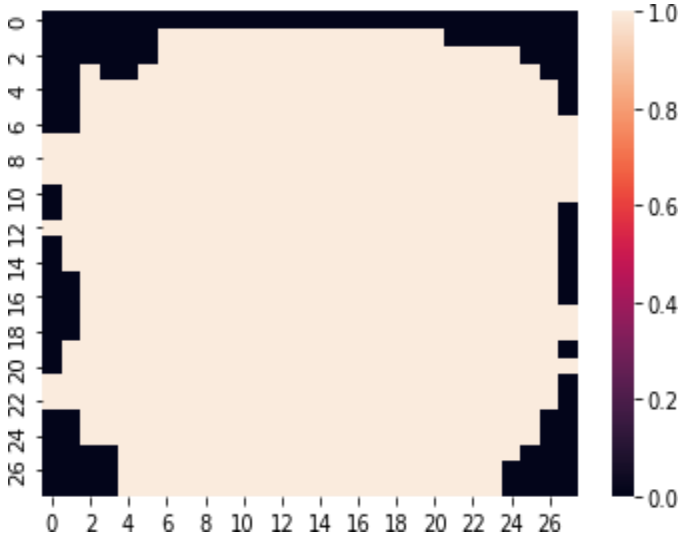
You should see the following output:

```
118 features have been removed, 666 features remain
```

8) We can also use the get_support function to see what features have been dropped. This returns a Boolean array which we can then visualise by reshaping it into a 28x28 array and using the seaborn heatmap function.

```
# We can use the get_support function to see which features have been
dropped

selected_features = variance_selector.get_support()

selected_features = selected_features.reshape(28,28)

# Visualise which pixels have been dropped

sns.heatmap(selected_features,cmap='rocket')

plt.show()
```

You should see the below plot:



The black pixels in the grid are those which have been dropped due to zero variance. Unsurprisingly, these are the pixels around the edges and the corners of the image, given that the digits are centered in the middle of the 28x28 image.

Part Three: Filter Methods Using SelectKBest

9) Scikit Learn also provides a selector called SelectKBest which can be used for filter-based feature selection. To use it, you need to specify a test statistic and a value for *k*, the number of features which you want to retain. Scikit Learn provides several different functions for calculating different test statistics; the most appropriate one will depend on the input variable (categorical or numeric) and whether it is a regression or classification task. In this case, we use the f-classif function, which computes the ANOVA F-value for the provided sample.

We follow the same steps as before, first instantiating the selector, and then using fit_transform to both fit and transform the training dataset. Finally, we use the transform function to drop the same features from the test set too.

In this case, we have set *k*=200. However, when applying feature selection to a dataset, you may want to explore different values of *k* to see which provides the best performance.

```
# Use the SelectKBest selector from sklearn to select the k features
with the best scores on a selected test statistic

from sklearn.feature_selection import SelectKBest, f_classif

selector = SelectKBest(f_classif, k=200)

X_train_fs = selector.fit_transform(X_train_fs, y_train)
X_test_fs = selector.transform(X_test_fs)
```

10) To visualise the features which have been removed now, we need to combine those that were removed by the VarianceThreshold selector and those removed by the SelectKBest selector. We do this by using the get_support function to return the indices of the features retained by the VarianceThreshold and then using the Boolean values from the second selector to identify the indices of the features retained by the second selector.

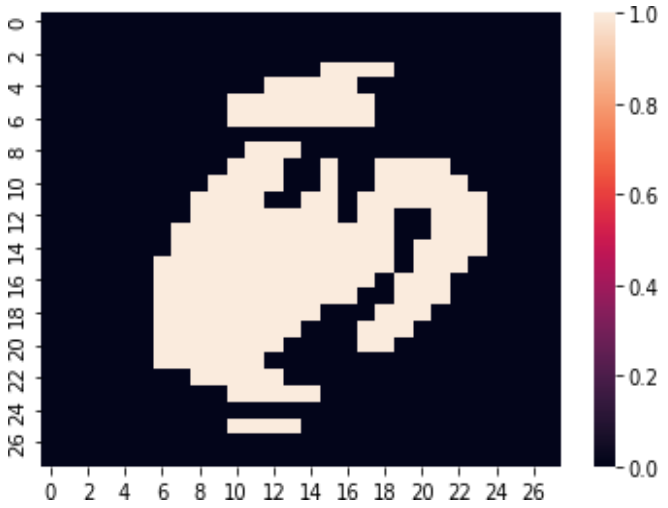
```
# Create boolean array for all features

new_features_indices = \
variance_selector.get_support(indices=True)[selector.get_support()]
```

```
new_features_boolean = np.isin(np.arange(784), new_features_indices)
```

As before we can then reshape the one-dimensional array into a 28x28 two dimensional array and visualise this using the seaborn heatmap function.

```
# Reshape and plot as a heatmap

sns.heatmap(new_features_boolean.reshape(28,28),cmap='rocket')


plt.show()
```

The cell output should look like the below.



The pixels which are shaded black correspond to those which we have dropped. Again, it's probably not too surprising to see that the most important features are the pixels near the centre of the image.

Part Four: Recursive Feature Elimination using RFECV

Recursive Feature Elimination is a wrapper-based feature selection algorithm. A wrapper-based feature selection algorithm uses a machine learning algorithm trained on the dataset to help select features. With Recursive Feature Elimination the model is first trained on the full dataset and then features are removed until the desired number remains.

Scikit Learn provides two selectors which implement Recursive Feature Elimination:

- **RFE** provides an implementation of Recursive Feature Elimination. When using this, you have to select the number of features you wish to retain.
- **RFECV** provides an implementation of Recursive Feature Elimination with Cross Validation. In this instance, the algorithm uses cross-validation instead to identify the best performing number of features.

In this workshop we will use RFECV. As this is a wrapper method, we also have to select a supervised learning estimator which is what the RFECV selector will use to understand feature importance. Not all supervised learning estimators will work here – we need one which provides information on feature importance, either through coefficients or a feature importance value. We will use the RandomForestClassifier here, although Decision Trees and Logistic Regression would also work.

11) Run the code below to run the required imports from Scikit Learn

```
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.feature_selection import RFECV
```

12) We then use the StandardScaler to normalise the data. This scales the data so that the mean of each variable is 0, and the standard deviation is 1.

```
# Standardise data before passing to model

scaler = StandardScaler()
X_train_fs = scaler.fit_transform(X_train_fs)
X_test_fs = scaler.transform(X_test_fs)
```

13) We then instantiate the selector and use the fit_transform and transform on the training dataset and test dataset respectively, as we did with the VarianceThreshold selector and the SelectKBest selector.

To save on running time we have set the step to five, which means features are removed five at a time. We have also set the number of cross-validation folds to three. (A value of five might be better here but this will take longer to run.)

NB – this will take a few minutes to run

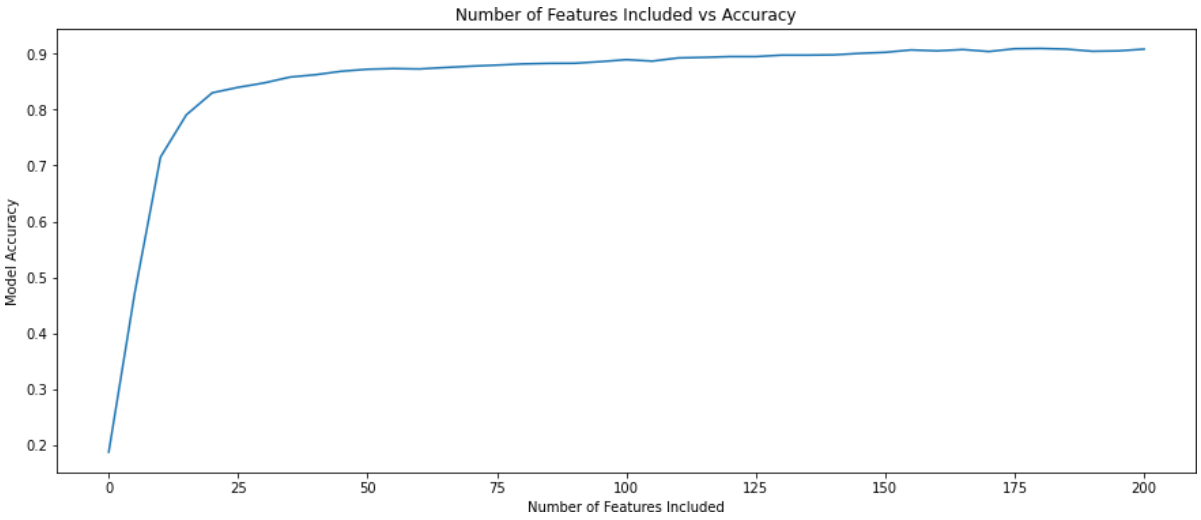
```
rf = RandomForestClassifier(random_state=0) # Use RandomForestClassifier as
the base model
rfecv = RFECV(rf, cv=3, step=5)

X_train_fs = rfecv.fit_transform(X_train_fs, y_train)
X_test_fs = rfecv.transform(X_test_fs)

print(f"Number of remaining features: {X_train_fs.shape[1]}")
```

We can see that in this case it has not removed many features, which is perhaps not surprising as we have already reduced the number of features quite significantly.

14) The fitted model also has a cv_results_ attribute which is a dictionary which contains the test scores from each fold of the cross-validation, the mean scores, and the standard deviation. We can use rfecv.cv_results_['mean_test_score'] to plot the model accuracy against the number of features, using the code below.

```
plt.figure( figsize=(15, 6))
plt.title('Number of Features Included vs Accuracy')
plt.xlabel('Number of Features Included')
plt.ylabel('Model Accuracy')
plt.plot(np.linspace(0,200,41), rfecv.cv_results_['mean_test_score'])
plt.show()
```

This should give you a plot like the one below. You can see that the accuracy reaches 80% with less than 25 features –that means that only 25 pixels from the image (3% of the available features) are required to accurately predict the digit more than 80% of the time! After that, adding additional features only leads to more incremental improvements – for example, to reach 90% model accuracy we need around 150 features.


Number of Features Included vs Accuracy
Model Accuracy
Number of Features Included
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.9
0
25
50
75
100
125
150
175
200

Part Five: Training and Evaluating a Model

Now we have used feature selection techniques to reduce the number of features in our dataset, we can train and evaluate a model on the new, transformed dataset as we normally would.

15) Here we instantiate a new RandomForestClassifier model, and then fit it to the data with the selected features.

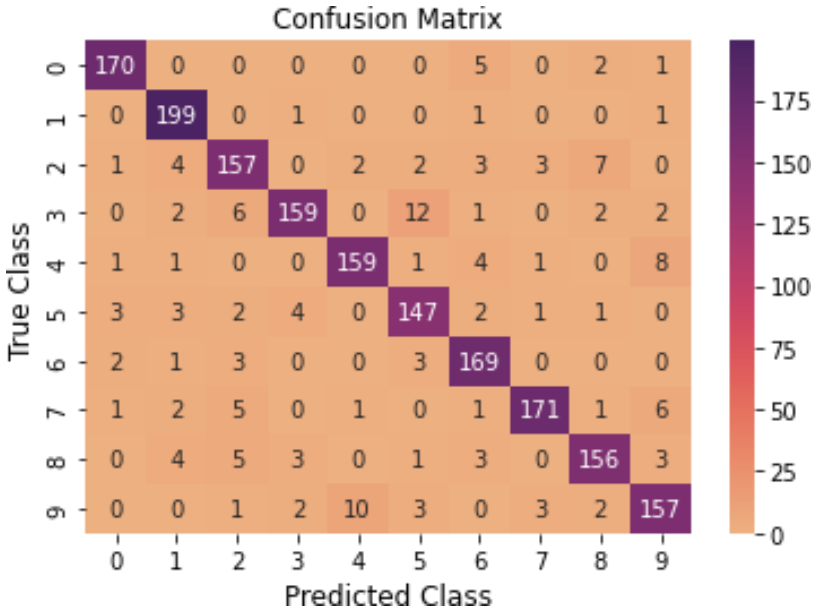
```
rf_selectedfeatures = RandomForestClassifier()

rf_selectedfeatures.fit(X_train_fs, y_train)
```

16) Having fitted the model, we can then use the predict() method to make predictions on the test dataset, and then use the accuracy_score and confusion_matrix functions to evaluate it. We use the seaborn heatmap function to visualise the confusion matrix.

```
# Make predictions on the test data

y_pred = rf_selectedfeatures.predict(X_test_fs)

print(f"Accuracy Score: {accuracy_score(y_test,y_pred)*100:.2f}%")

cm = confusion_matrix(y_test,y_pred)

ax = sns.heatmap(cm, cmap='flare',annot=True, fmt='d')

plt.xlabel("Predicted Class",fontsize=12)
plt.ylabel("True Class",fontsize=12)
plt.title("Confusion Matrix",fontsize=12)
plt.show()
```

You should get a cell output similar to the below (the accuracy you get may vary slightly from the below):

```
Accuracy Score: 91.33%
```



So, with less than a quarter of the features in the training data, we have achieved a model accuracy of over 90%.

Part Six: Addressing Class Imbalance

In the workshop last week, we addressed class imbalance by using the class_weight argument when training the model. In this workshop, we are going to briefly revisit this dataset to demonstrate resampling methods that we can use when we have class imbalance.

In your own time, if you wish to do so, you can go back to the code from last week and try running this code again without using the class_weight argument when training the model. How does this affect the overall accuracy? How does this affect the precision and recall for the three different classes? Which of the two models would you recommend using for the given scenario?

There are other ways we can address class imbalance. One of the main ways we can do this is through **resampling:**

- **Oversampling:** This method takes the minority class and randomly duplicates occurrences until it matches the frequency of the most commonly occurring target class. This has the benefit of not losing any data, but because you are duplicating records, you run the risk of reinforcing associations which would not exist in real, unduplicated data. For example, oversampling could duplicate outliers in the data.

- **Undersampling:** This method takes the most commonly occurring target event, and randomly removes occurrences until the frequency matches the frequency of the minority target class. This has the benefit of staying true to real data by not creating any artificial duplicates, but at the cost of possibly losing a lot of data.

Typically, we perform oversampling or undersampling on the training dataset **only**, so that the test dataset still reflects the actual distribution that you would see in any future real-world data you gathered. In addition, because oversampling involves duplicating occurrences in the data, doing this before splitting it would mean that the **same** duplicated occurrences could end up in both the test and the training dataset. The test dataset is therefore no longer truly 'unseen' data and won't give you an accurate

reflection of likely performance in the real-world – this problem is known as 'data leakage'.

Because the class distribution remains unbalanced in the test dataset, you will need to choose an appropriate metric to judge model performance, as overall model accuracy is unlikely to be the right choice. For example, consider Precision, Recall, AUC ROC, etc.

We can implement both oversampling and undersampling using the Imbalanced Learn library. First, open a new notebook and use the command below to install the library:

```
pip install -U imbalanced-learn
```

Then restart the Kernel from the top menu

Now run the command below to run the required imports:

```
In [3]: import numpy as np
        import pandas as pd
        import imblearn
        import matplotlib.pyplot as plt
        import seaborn as sns
        %matplotlib inline
```

The file you need to upload for this section is saved on Blackboard as **Cardiotocographic.csv** (This is the same file we used for the workshop last week.)

Having uploaded the dataset to the same directory as your notebook, we can now use the read_csv function to read it into a pandas dataframe. We can then inspect the first and last rows.

```
cardio_data = pd.read_csv('Cardiotocographic.csv')

cardio_data.head()

cardio_data.tail()
```

```
cardio_data = pd.read_csv('Cardiotocographic.csv')

cardio_data.head()
```

	BPM	APC	FMPS	UCPS	DLPS	SDPS	PDPS	ASTV	MSTV	ALTV	MLTV	Width	Min	Max	NSP
0	120	0.000000	0.0	0.000000	0.000000	0.0	0.0	73	0.5	43	2.4	64	62	126	2
1	132	0.006380	0.0	0.006380	0.003190	0.0	0.0	17	2.1	0	10.4	130	68	198	1
2	133	0.003322	0.0	0.008306	0.003322	0.0	0.0	16	2.1	0	13.4	130	68	198	1
3	134	0.002561	0.0	0.007682	0.002561	0.0	0.0	16	2.4	0	23.0	117	53	170	1
4	132	0.006515	0.0	0.008143	0.000000	0.0	0.0	16	2.4	0	19.9	117	53	170	1

```
[5] cardio_data.tail()
```

	BPM	APC	FMPS	UCPS	DLPS	SDPS	PDPS	ASTV	MSTV	ALTV	MLTV	Width	Min	Max	NSP
2121	140	0.000000	0.000000	0.007426	0.0	0.0	0.0	79	0.2	25	7.2	40	137	177	2
2122	140	0.000775	0.000000	0.006971	0.0	0.0	0.0	78	0.4	22	7.1	66	103	169	2
2123	140	0.000980	0.000000	0.006863	0.0	0.0	0.0	79	0.4	20	6.1	67	103	170	2
2124	140	0.000679	0.000000	0.006110	0.0	0.0	0.0	78	0.4	27	7.0	66	103	169	2
2125	142	0.001616	0.001616	0.008078	0.0	0.0	0.0	74	0.4	36	5.0	42	117	159	1

We can also explore the data:

```
cardio_data.describe()
```

```
[6] cardio_data.describe()
```

	BPM	APC	FMPS	UCPS	DLPS	SDPS	PDPS	ASTV	MSTV	ALTV	
count	2126.000000	2126.000000	2126.000000	2126.000000	2126.000000	2126.000000	2126.000000	2126.000000	2126.000000	2126.00000	212
mean	133.303857	0.003170	0.009474	0.004357	0.001885	0.000004	0.000157	46.990122	1.332785	9.84666	
std	9.840844	0.003860	0.046670	0.002940	0.002962	0.000063	0.000580	17.192814	0.883241	18.39688	
min	106.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	12.000000	0.200000	0.00000	
25%	126.000000	0.000000	0.000000	0.001876	0.000000	0.000000	0.000000	32.000000	0.700000	0.00000	
50%	133.000000	0.001630	0.000000	0.004482	0.000000	0.000000	0.000000	49.000000	1.200000	0.00000	
75%	140.000000	0.005631	0.002512	0.006525	0.003264	0.000000	0.000000	61.000000	1.700000	11.00000	
max	160.000000	0.019284	0.480634	0.014925	0.015385	0.001353	0.005348	87.000000	7.000000	91.00000	

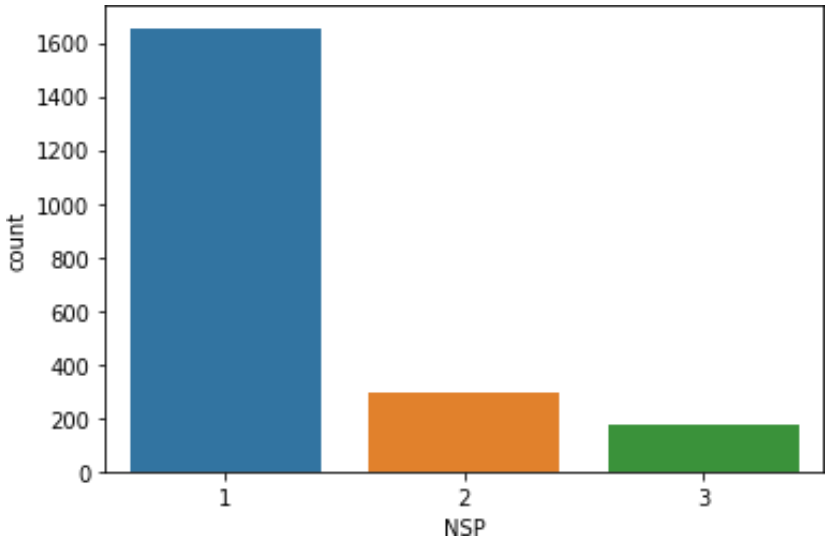
```
cardio_data['NSP'].value_counts()
```

```
cardio_data['NSP'].value_counts()
```

```
1    1655
2     295
3     176
Name: NSP, dtype: int64
```

We can see we have imbalanced classes with 77.8% of the observations belong to the Normal class. We can also use the seaborn countplot to visualise this.

```
sns.countplot(cardio_data, x="NSP")
plt.show()
```



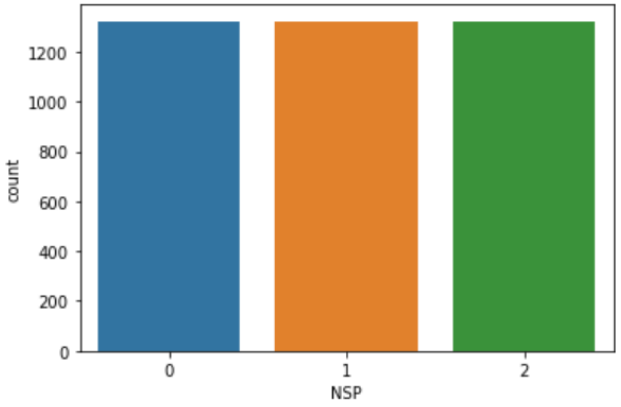
Next, we are going to use the train_test_split from Scikit Learn to divide the data into a training dataset and a test dataset.

We also deduct 1 from the values of the class labels in the NSP column. This is because Keras assumes our class labels start at 0, whereas for this dataset the class labels are 1, 2 and 3. Once we have deducted 1, class 0 is Normal, class 1 is Suspect, and class 2 is Pathologic.

```
from sklearn.model_selection import train_test_split

X = cardio_data.drop('NSP',axis=1)
y = cardio_data['NSP'] -1

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2,
stratify=y, random_state=0)
```

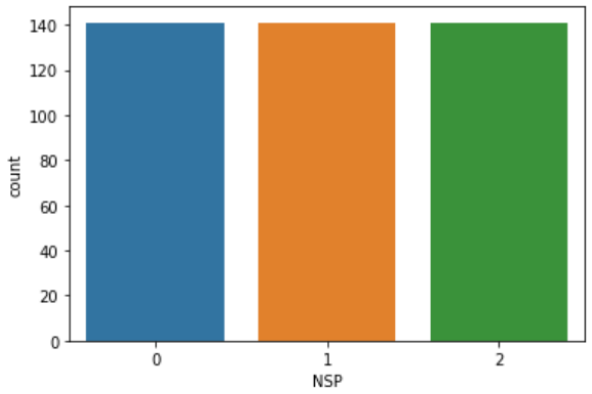
To carry out oversampling, we can use RandomOverSampler. You can run the code below to carry out random oversampling. We can then use a countplot to visualise the results.

```
from imblearn.over_sampling import RandomOverSampler
resampler=RandomOverSampler(random_state=0)
X_train_oversampled,y_train_oversampled=resampler.fit_resample(X_train,y_train)
sns.countplot(x=y_train_oversampled)
plt.show()
```



To carry out undersampling we follow the same process, except in this case we want to undersample classes 0 and 1. We use the RandomUnderSampler to achieve this.

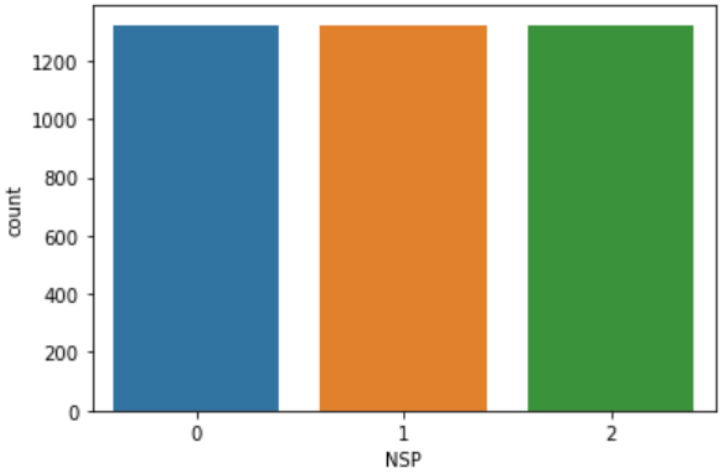
```
from imblearn.under_sampling import RandomUnderSampler
resampler=RandomUnderSampler(random_state=0)
X_train_undersampled,y_train_undersampled=resampler.fit_resample(X_train,y_train)
sns.countplot(x=y_train_undersampled)
plt.show()
```



Finally, we can also implement SMOTE as an alternative to random over sampling:

In [13]:
```
from imblearn.over_sampling import SMOTE

resampler = SMOTE(random_state=0)
X_train_smote, y_train_smote = resampler.fit_resample(X_train, y_train)

sns.countplot(x=y_train_smote)
```

Out[13]:
```
<AxesSubplot:xlabel='NSP', ylabel='count'>
```



If you have time, you can try re-running the code from last week with the oversampled and undersampled training datasets. Which gives you the best results? (Hint – don't forget to remove the class weights argument when fitting the model as you are no longer using an imbalanced dataset.)